

Reinforcement Learning Assignment

Martina Olivato, Angelina Podolako, Tom Philipsen, Lucas Maarse

3rd April 2025

1 Introduction

The aim of this project is to implement and evaluate various Reinforcement Learning (RL) algorithms in a given environment, and to compare their performance. In the context of Reinforcement Learning, the "agent-environment loop" defines the interaction between the agent and the environment. At each time step, the agent observes the current state of the environment, selects an action based on this observation, and receives a reward from the environment. Subsequently, the environment transitions to a new state, and the process repeats until the agent reaches a terminal state.

For this project, the environment is the "*Frozen Lake*", a grid world where the agent must navigate a lake with frozen patches and holes, aiming to reach a goal while avoiding obstacles. The agent has four possible actions at each step: up, down, left, and right. We tested all algorithms in a stochastic environment by setting 'is_slippery=true'.

To investigate the effectiveness of different RL approaches, we implemented and tested algorithms from three major categories: *Dynamic Programming*, *Monte Carlo*, and *Temporal Difference* methods. Then, we compared and evaluated the performance of these algorithms based on several metrics, such as convergence speed and policy quality by running experiments and analyzing the results in terms of their learning efficiency and ability to solve the "*Frozen Lake*" problem.

2 Dynamic Programming algorithms

Dynamic Programming (DP) is a method of solving complex problems by breaking them down into smaller, simpler sub-problems and storing their solutions. Dynamic Programming methods are particularly useful for solving Markov Decision Processes (MDPs), enabling the computation of optimal policies and value functions.

Two common algorithms used in Dynamic Programming are *Policy iteration* and *Value Iteration*. While both algorithms ultimately find the optimal policy and value function, they differ in how they approach the problem.

Policy iteration alternates between policy evaluation and policy improvement. The process works as follows:

1. Initialize a random policy π and the value function $V(s)$
2. Policy Evaluation: compute $V_\pi(s)$ for all states using the Bellman Expectation Equation:

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma]$$
This step estimates how good the policy is and is repeated until the value function converges.
3. Policy Improvement: improve the policy greedily with respect to the current value function: $\pi'(s) = \arg \max_a P(s'|s, a) [R(s, a, s') + \gamma]$
If the policy changes, go back to Policy Evaluation (point 2).
If the policy is stable, then stop and return $V \approx v^*$ and $\pi \approx \pi^*$.

Value Iteration iterates over each state and updates the value function based on the Bellman Optimality Equation:

1. Initialize $V(s)$ arbitrarily.
2. Value Update (Bellman Optimality Equation): for each state s , update the value function using: $V(s) \leftarrow \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$
Repeat until convergence.
3. Extract the Policy: once values are stable, derive the optimal policy:

$$\pi^*(s) = \arg \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$$

In Policy Iteration, the process alternates between policy evaluation and policy improvement, converging when the policy stabilizes. In contrast, Value Iteration combines both steps into a single one by directly applying the Bellman Optimality Equation for each state, repeating until the value function converges and extracts the optimal policy. Policy Iteration generally converges faster in terms of policy improvement, but can be computationally expensive. On the other hand, Value Iteration may take more iterations to converge, but is more efficient since it avoids separate policy evaluation and improvement phases.

In our experiments, we analyzed the impact of the discount factor γ and convergence threshold θ . As both algorithms return an optimal policy w.r.t. the convergence threshold, θ , we used the metric of convergence speed to analyze the algorithms with the different hyper-parameters. The convergence speed reflects how efficiently an algorithm arrives at an optimal policy. The faster it converges, the better. The results show that in our environment both converge roughly evenly quick. We also see the trend of a higher γ converging faster in both algorithms. Finally, in both algorithms, we see that when increasing the θ , convergence slows down. This happens until roughly $\theta = 0.4$ where it plateaus.

3 Monte Carlo algorithms

Monte Carlo methods estimate value functions by learning from experience, using observed sequences of states, actions, and rewards. The expected return is

estimated by averaging observed returns, which converges to the true expectation as more samples are collected. Monte Carlo methods fall under Generalized Policy Iteration, alternating between policy evaluation (prediction) and policy improvement (control).

In this project, we implemented both the *MC Prediction* and *MC Control with Exploring Starts* algorithms as separate functions. In our *Monte Carlo Exploring Starts* implementation, we estimate the state-action value function $q_\pi(s, a)$ using First-visit Monte Carlo. The algorithm proceeds as follows:

1. Initialize the action-value function $Q(s, a)$ arbitrarily.
2. Start each episode from a random state-action pair.
3. Generate an episode by following a policy.
4. Compute the return for each state-action pair.
5. Update $Q(s, a)$ using the average of returns:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(G - Q(s, a))$$
6. Improve the policy by choosing the action with the highest Q -value (greedy policy).
7. Repeat until convergence.

Monte Carlo Exploring Starts ensures full exploration by randomly selecting start states. We evaluated Monte Carlo Control using different values of γ . The results indicate that a lower γ produces better results early on, but a higher γ produces the better overall result.

The *MC Prediction* algorithm is generally the same, except that instead of estimating the state-action value function, we only estimate the state value function and do not update the policy. We decided to use this approach of only estimating the state value function, as we intend to use this function later on when comparing the algorithms of MC Control ES, SARSA and Q-Learning. There we compare the algorithms using the estimated value function, calculated using MC Prediction, per state.

4 Temporal Difference Learning

Temporal Difference (TD) Learning combines elements of Dynamic Programming and Monte Carlo methods by updating value estimates incrementally after each step, rather than waiting until the end of an episode. Unlike Monte Carlo methods, which require complete episodes to update values, TD methods use bootstrapping, making it more suitable for both episodic and continuing tasks. TD methods can be applied in both on-policy and off-policy settings, leading to algorithms such as *SARSA* (on-policy) and *Q-learning* (off-policy).

SARSA updates the action-value function while following the current policy. The algorithm proceeds as follows:

1. Initialize the Q -table for all states and actions.
2. Start an episode by observing the initial state s .
3. Choose an action a using an ϵ -greedy policy.
4. Take the action, observe the reward r and the next state s' .
5. Select the next action a' from s' using the same ϵ -greedy policy.
6. Update the Q -value using the formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$
7. Repeat until the episode ends.

Q-Learning, on the other hand, learns the optimal action-value function $Q(s, a)$ by maximizing future rewards, independent of the policy being followed. This makes it an off-policy method. The algorithm follows the steps:

1. Initialize the Q -table for all states and actions.
2. Start an episode by observing the initial state s .
3. Choose an action a using an ϵ -greedy policy.
4. Take the action, observe the reward r and the next state s' .
5. Update the Q -value using the formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$
6. Set $s \leftarrow s'$ and repeat until the episode ends.

Despite their similarities, they differ in how they learn from the environment and update the action-value function. The key difference lies in how each algorithm updates its value estimates. SARSA, as an on-policy algorithm, updates its action-value function using the action actually chosen by the policy it is following. This means it learns the value of the policy it is actively using to make decisions. In contrast, Q-learning is an off-policy algorithm that updates its action-value estimates based on the action that maximizes future rewards, regardless of the action taken by its behavior policy. This allows Q-learning to learn the optimal policy while following a different behavior policy. Another distinction is in how these algorithms balance exploration and exploitation. SARSA explicitly accounts for the exploration strategy by updating values using the next action selected under its current policy. This behavior leads to a more cautious learning process, where the algorithm considers both exploratory and greedy actions during updates. On the other hand, Q-learning always updates towards the action with the highest predicted reward, emphasizing exploitation. While this aggressive approach can accelerate learning of the optimal policy, it can also make Q-learning more prone to risky behavior in environments where exploration is crucial. In our implementation of the algorithms, we chose to use a decaying ϵ , ensuring early exploration with gradual exploitation. This

approach guarantees optimality among the epsilon-greedy policies. To better understand the differences between SARSA and Q-learning, we conducted several experiments where we studied the learning behavior of the two algorithms by plotting the evolution of Q-values over time, using varying hyperparameters.

Here, we found that for both SARSA and Q-learning, a lower α performs generally better than higher α values. Furthermore, a γ closer to 1 offers a higher rolling average reward. Finally, we found no significant difference in the values for the initial ϵ . Initial ϵ values of above 0.1 perform overall very similar.

5 Comparison and Discussion

In this project, we compared the policies acquired by different algorithms: Random Policy, Monte Carlo Control with Exploring Starts, Sarsa, Q-Learning. The random policy served as a baseline for performance comparison. We evaluated these policies with the use of the Monte Carlo Prediction algorithm, which estimates the value function based on sampled episodes. We evaluated these policies using the Monte Carlo Prediction algorithm, which estimates the value function based on sampled episodes. In our experiments, we used 1000 episodes per algorithm. While using more episodes would have yielded more accurate estimates, it would also have increased computational cost.

All algorithms significantly outperformed the random policy, particularly in states far from the goal, confirming the effectiveness of our implementations. SARSA and Q-Learning produced similar value estimates; however, Q-Learning exhibited lower estimated values in areas farther from the 'holes'. This is likely due to Q-Learning's more aggressive, off-policy nature, which can result in riskier behavior—an important factor in our environment, where the goal state is located near dangerous 'hole' states. We also observed that MC Control assigns relatively low values to states far from the goal, indicating weaker generalization from those distant states.

Based on this final comparison and the earlier results regarding hyperparameters, we draw the following conclusions: Each algorithm has its own strengths and limitations. When full knowledge of the environment's dynamics is available, using Dynamic Programming methods—such as Policy Iteration or Value Iteration—is most sensible, as these algorithms are guaranteed to converge to the optimal policy.

In environments where the dynamics are unknown, one must rely on either Monte Carlo Control or Temporal Difference (TD) methods. If the environment is continuous or does not have clear episode boundaries, TD methods are generally more suitable. In episodic environments, both MC and TD approaches

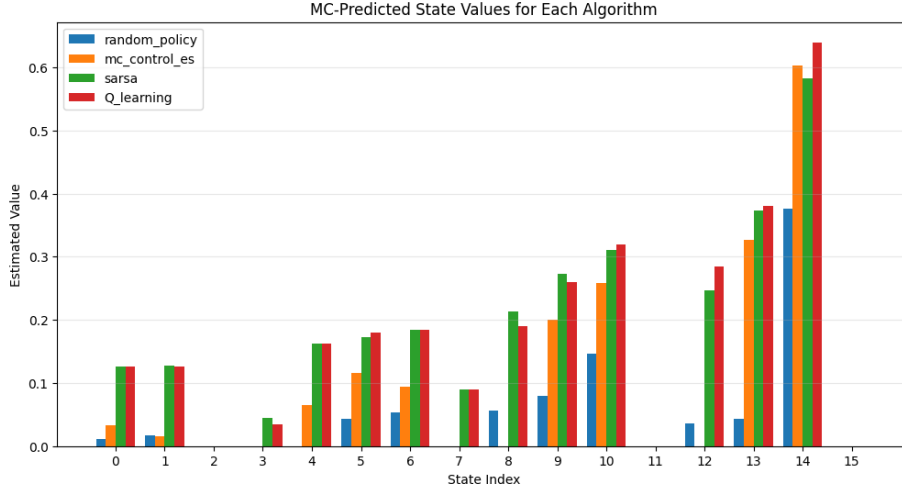


Figure 1: Comparison of algorithms using the estimated value function at each state, calculated by MC Prediction.

can be used effectively, although neither is guaranteed to find the optimal policy.

For detailed insights into the best hyperparameter settings, we refer the reader to the earlier sections of this report corresponding to each algorithm. The appendix includes the full results from our experiments, upon which these conclusions are based.

6 Conclusion

In conclusion, this project provided an insight into three important Reinforcement Learning algorithms and tested them with different hyper-parameters to better understand the strengths and weaknesses of the algorithms. Dynamic programming algorithms require a complete model of the environment and are computationally expensive but guarantee convergence to optimal policies. Monte Carlo methods do not require a model and work well with episodic tasks but need long episodes for accurate value estimation. Temporal Difference methods balance efficiency and sample complexity by learning from incomplete episodes but can be more sensitive to learning rates and exploration strategies.

Overall, all the algorithms show to have a better performance for high values of the hyper-parameters, suggesting that the agent is learning fast from their experiences and values future rewards more. In deterministic environments, lower ϵ might work well, but if the environment has randomness (e.g., stochastic transitions in FrozenLake), high exploration helps avoid getting stuck in suboptimal strategies. This also suggests that the environment likely benefits

from aggressive learning and long-term planning.

A Policy and Value Iteration



Figure 2: Environment for testing Policy and Value Iteration

Final Policy Heatmap

0	Left	Left	Right	Right	Left	Left	Right	Right	Right	Left	Left	Left
1	Down	Down	Up	Right	Down	Left	Left	Right	Right	Left	Left	Left
2	Left	Left	Left	Right	Down	Down	Down	Up	Up	Left	Left	Down
3	Down	Down	Down	Down	Down	Left	Left	Left	Left	Right	Down	Down
4	Down	Down	Up	Right	Up	Left	Left	Left	Left	Down	Down	Down
5	Down	Left	Left	Left	Left	Right	Left	Left	Right	Down	Down	Down
6	Up	Down	Down	Down	Down	Down	Down	Down	Up	Down	Left	Up
7	Left	Right	Right	Right	Right	Right	Down	Left	Left	Right	Left	Left
8	Down	Up	Up	Right	Right	Right	Right	Up	Down	Left	Left	Left
9	Up	Left	Left	Right	Up	Up	Left	Left	Right	Up	Down	Down
10	Up	Right	Down	Left	Left	Left	Right	Down	Left	Left	Right	Down
11	Right	Right	Right	Left	Left	Left	Left	Right	Right	Down	Down	Right
	0	1	2	3	4	5	6	7	8	9	10	11
	X-axis (States)											
Y-axis (States)	0	1	2	3	4	5	6	7	8	9	10	11

Figure 3: Environment for testing Policy and Value Iteration

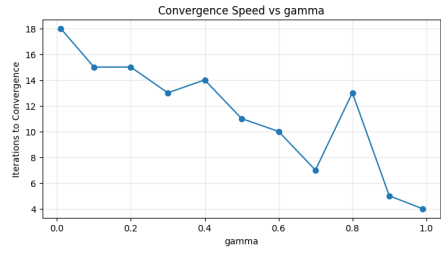


Figure 4: Policy Iteration: Convergence speed vs Gamma

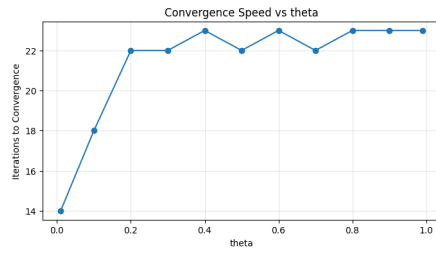


Figure 5: Policy Iteration: Convergence speed vs Theta

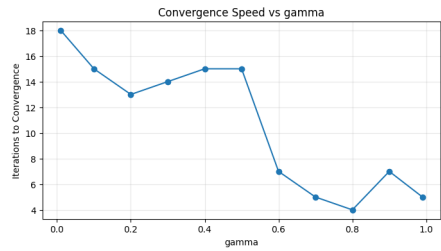


Figure 6: Value Iteration: Convergence speed vs Gamma

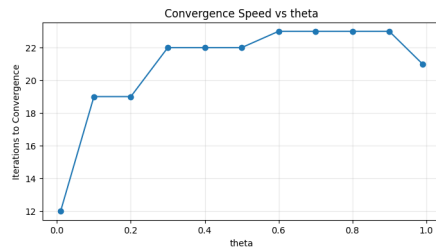


Figure 7: Value Iteration: Convergence speed vs Theta

A Monte Carlo and Temporal Difference

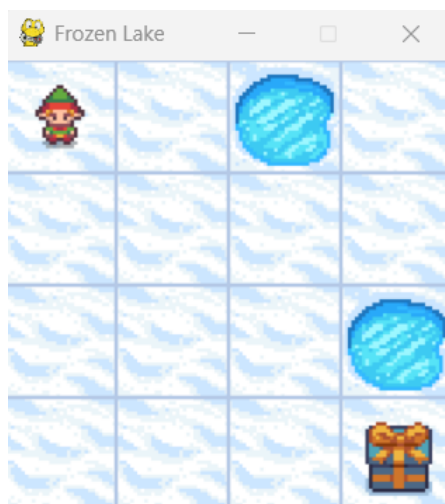


Figure 8: Environment for testing MC and TD

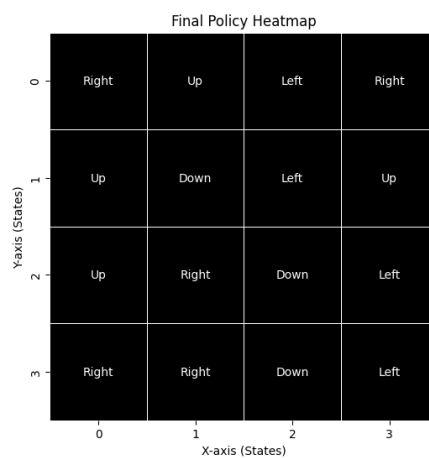


Figure 9: Final Policy for MC Control

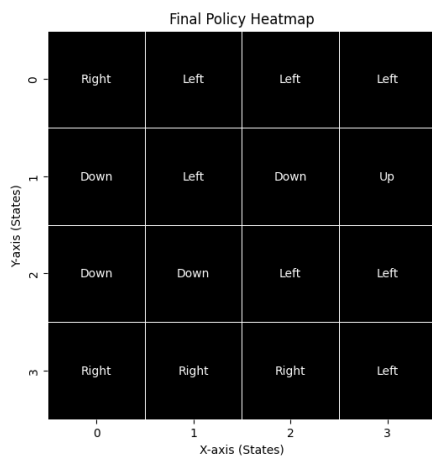


Figure 10: Final Policy for Sarsa

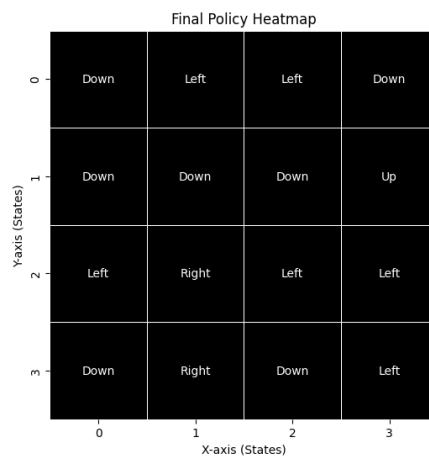


Figure 11: Final Policy for Q-Learning

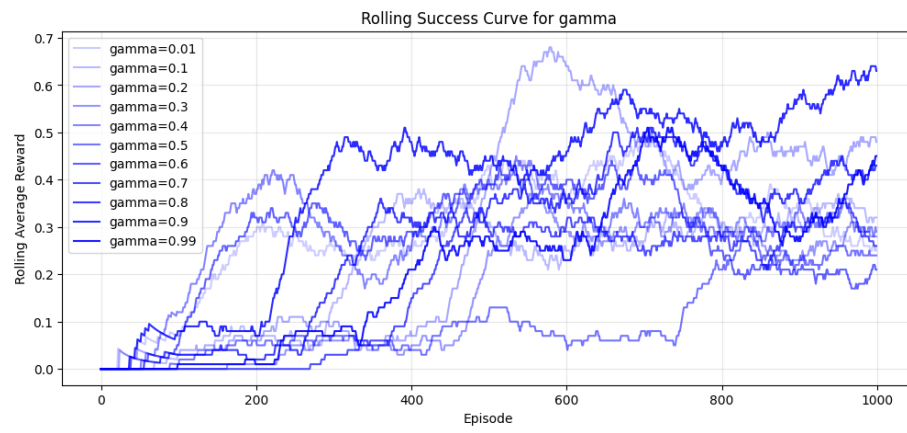


Figure 12: MC ES - Rolling Success for Gamma

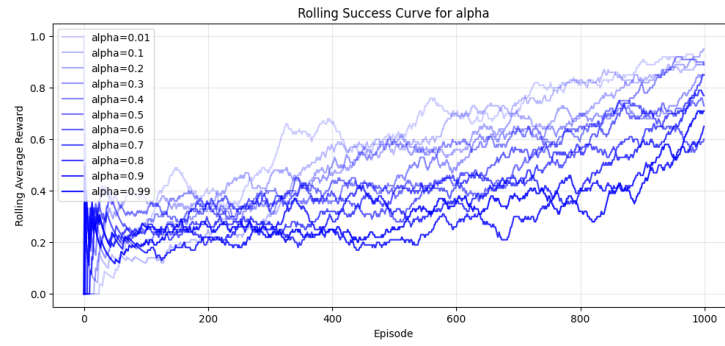


Figure 13: SARSA - Rolling Success for alpha

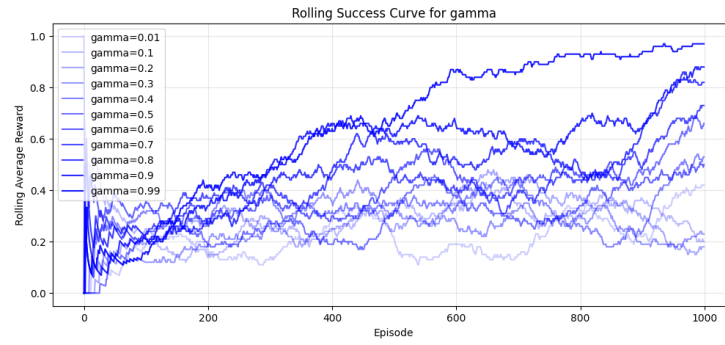


Figure 14: SARSA - Rolling Success for gamma

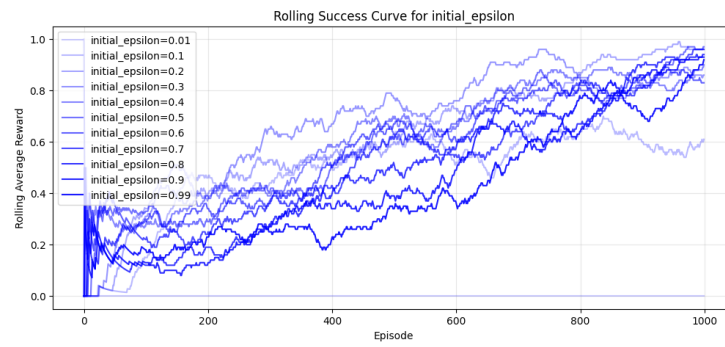


Figure 15: SARSA - Rolling Success for initial epsilon

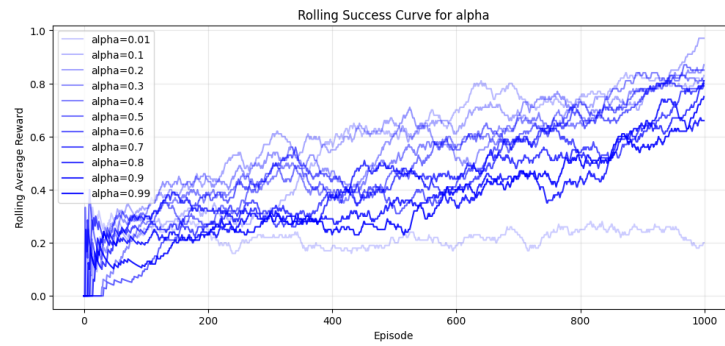


Figure 16: Q-Learning - Rolling Success for alpha

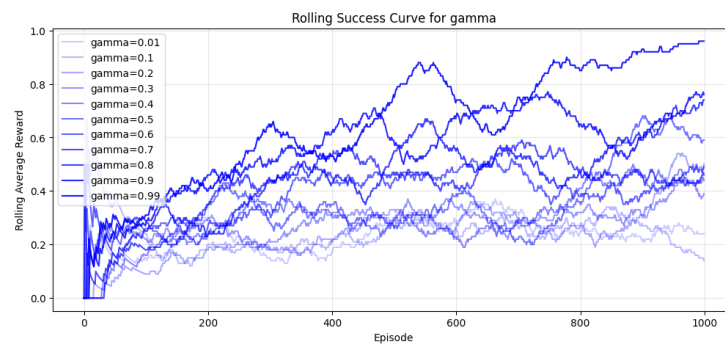


Figure 17: Q-Learning - Rolling Success for gamma

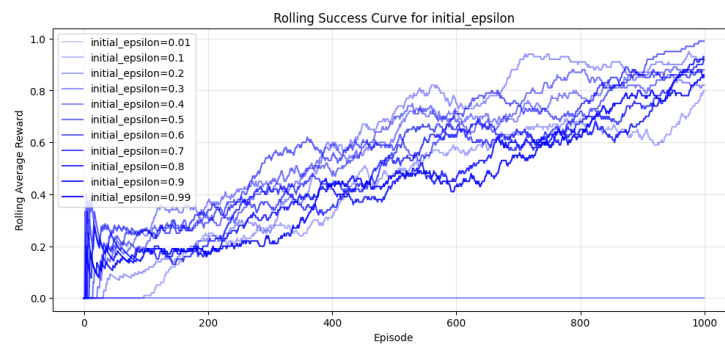


Figure 18: Q-Learning - Rolling Success for initial epsilon